ShopIndia — Multi-Panel Operations & Dispatch Architecture

Version: 2.0.0
Target Scope: Full Multi-Channel Fulfillment

Rider Panel • Regional Admin Panel • Shiprocket 3PL • Local Quick-Commerce

APPROVED FOR IMPLEMENTATION

1. Order Fulfillment Taxonomy & Shiprocket Clarity

ShopIndia categorizes all marketplace orders into three discrete fulfillment pathways, giving both local merchants and operations managers clarity on dispatch mechanics:

📦 Stream A: Normal Delivery

Channel: Shiprocket (3PL Logistics)

Timeline: 2–5 Business Days

Carriers: Bluedart, Delhivery, DTDC, Shadowfax

Execution: Inter-city / interstate parcels shipped with auto-generated AWB and labels.

🚀 Stream B: Quick Commerce

Channel: ShopIndia Rider Fleet

Timeline: 10–30 Minutes

Radius: 10 km to 100 km (rider slider)

Execution: Rider drives to Vendor store, verifies package checklist, and delivers to customer doorstep with 4-digit OTP.

🔧 Stream C: Services & HVAC

Channel: Technician / Service Partner

Timeline: Scheduled / Immediate

Scope: AC Repair, Installation, Diagnostics

Execution: Certified technicians dispatched to customer site with work verification OTP.

Key Architectural Decision — Shiprocket API Credentials:
Super Admin holds the Master Shiprocket Account keys centrally. Each vendor's store address is registered via API as a *Shiprocket Pickup Location*. When a vendor has a Normal Delivery, they do NOT need their own Shiprocket account—they simply click `[ 📦 Ship via Shiprocket ]`, and the platform automatically fetches the best courier rate, generates the AWB, and creates the printable shipping label. Both Regional Admins and Super Admin have oversight to track manifests and delays.

2. Platform Portals & Unified Routing

Portal Name	URL Route	Primary Users	Key Responsibilities
Customer Storefront	`/`	Shoppers & Service Clients	Product search, quick commerce ordering, service booking, real-time tracking, delivery OTP.
Super Admin Portal	`/admin`	Executive & Ops Admins	Master governance, Regional Admin approvals, platform-wide rider KYC, categories, finance, Shiprocket keys.
Regional Admin Portal	`/regional-admin` `/regional-admin/register`	City & District Regional Admins	Regional vendor oversight, regional rider approval, live delivery radar map, dispatch delay escalations.
Vendor Merchant Portal	`/vendor`	Store Owners & Merchants	Catalog, order packing, one-click `[ 🚀 Broadcast to Riders ]` or `[ 📦 Ship via Shiprocket ]`.
Rider / Delivery Portal	`/rider` `/rider-login`	Local Delivery Partners	Online/Offline toggle, 30s broadcast radar chime, atomic accept, 4-stage turn-by-turn route, instant wallet payout.

3. Regional Admin Module: Architecture & Governance

To support high-velocity regional scale across Indian cities and districts, the Regional Admin acts as the local operations lead:

- **Self-Service Onboarding:** Prospective regional managers apply at `/regional-admin/register`, providing Aadhaar/PAN ID proofs, contact details, and their target District/Zone.
- **Super Admin Approval:** Visible under `/admin` in the **"Regional Admins"** management tab. Super Admin reviews credentials, sets status to `approved`, and assigns branch/district boundaries.
- **Regional Scoped Dashboard:** Once authenticated, Regional Admins see only merchants, delivery riders, and orders active within their assigned territory.
- **Live Operations Radar:** Interactive Mapbox view tracking in-flight riders traveling from vendor stores to customer dropoffs, with emergency reassignment capability if a rider stalls.

4. Quick Commerce Delivery: 4-Stage Pipeline



5. Broadcast Radar & Atomic Concurrency Engine

When an order is broadcasted by a vendor, an alert chime plays on all online riders' devices within the vendor's GPS radius (10–100 km). Multiple riders may tap "Accept" simultaneously. Concurrency is resolved via atomic database locking:

```
-- Atomic First-Come-First-Served Order Claim in PostgreSQL
UPDATE deliveries
SET rider_profile_id = $1, status = 'assigned', assigned_at = NOW(), updated_at = NOW()
WHERE id = $2 AND status = 'broadcasted'
RETURNING *;
```

- **Winner (Row Count = 1):** Status becomes `assigned`. Modal dismisses and immediately opens the Turn-by-Turn active delivery interface.
- **Follower (Row Count = 0):** The server responds with `{ "success": false, "code": "ALREADY_ASSIGNED" }`. The UI shows a friendly toast alert and smoothly animates the card away.

6. Database Schema Enhancements (Prisma / PostgreSQL)

```
// server/prisma/schema.prisma (Core Models)

enum DeliveryStatus {
  broadcasted       // Available in pool for local riders
  assigned          // Claimed by rider, traveling to vendor
  arrived_pickup    // Reached merchant store
  picked_up         // Package collected, traveling to customer
  arrived_customer  // At customer location
  delivered         // Verified via 4-digit OTP or photo
  cancelled
}

model RegionalAdminProfile {
  id               String             @id @default(uuid())
  userId           String             @unique
  user             User               @relation(fields: [userId], references: [id], onDel
  branchId         String?
  regionName       String             // e.g. "Hyderabad Central"
  district         String?
  state            String?
  assignedPincodes String[]           @default([])
  status           RegionalAdminStatus @default(pending_approval)
  createdAt        DateTime           @default(now())
  @@map("regional_admin_profiles")
}

model RiderProfile {
  id              String        @id @default(uuid())
  userId          String        @unique
  user            User          @relation(fields: [userId], references: [id], onDelete: Ca
  vehicleType     String        @default("bike")
  vehicleNumber   String?
  licenseNumber   String?
  licenseUrl      String?
  aadhaarNumber   String?
  aadhaarUrl      String?
```

```
  panNumber       String?
  panUrl          String?
  serviceRadiusKm Int           @default(25) // 10km to 100km slider
  isOnline        Boolean       @default(false)
  walletBalance   Float         @default(0)
  deliveries      Delivery[]
  payouts         RiderPayout[]
  @@map("rider_profiles")
}

model Delivery {
  id              String         @id @default(uuid())
  orderId         String         @unique
  order           Order          @relation(fields: [orderId], references: [id], onDelete: C
  riderProfileId String?
  riderProfile    RiderProfile?  @relation(fields: [riderProfileId], references: [id])
  status          DeliveryStatus @default(broadcasted)
  deliveryFee     Float          @default(50)
  distanceKm      Float          @default(3.5)
  deliveryOtp     String?        // 4-digit verification code
  pickupName      String
  pickupAddress   String
  dropoffName     String
  dropoffAddress String
  @@map("deliveries")
}
```

7. Implementation Roadmap

- **Phase 1: Database Schema & Migration** — Update `schema.prisma` with `RegionalAdminProfile`, `RiderProfile`, `Delivery`, and `ShiprocketShipment`. Push to PostgreSQL RDS.
- **Phase 2: Backend REST APIs** — Implement Rider auth, atomic acceptance queue, stage progression, Regional Admin onboarding, and Shiprocket dispatch helper.
- **Phase 3: Rider Portal Frontend** — Build mobile-first `/rider` with Mapbox registration, 30s countdown radar chime, swipeable `SlideToConfirm`, and 4-digit OTP completion.
- **Phase 4: Regional Admin Portal** — Build `/regional-admin` with zone application, vendor monitoring, rider KYC approvals, and live delivery map.
- **Phase 5: Vendor & Super Admin Integrations** — Add `[ 🚀 Broadcast to Riders ]` and `[ 📦 Ship via Shiprocket ]` in `VendorOrders.tsx`. Add `RegionalAdminsPage.tsx` in `/admin`.